

پرسش ۱ (۲ نمره) — استفاده از متغیرهای سراسری می‌تواند مشکلاتی ایجاد کند. به دو نمونه از این مشکلات اشاره کنید.
(۱) دشواری فهم کارکرد بخش‌های مختلف برنامه (۲) سخت شدن استفاده مجدد از توابع

پرسش ۲ (۴ نمره) — در برنامه زیر به دست کم چهار خطای کامپایل اشاره کنید و به طور مختصر توضیح دهید:

```
#include <iostream>
#include <vector>
using namespace std;
```

```
int twice(int& x) {
    return x * 2;
}
```

تایپ بازگشتی تابع main باید int باشد

```
void main() {
    vector<vector<int>> v(10, 10 ); نمی‌تواند برای مقداردهی اولیه برداری از اعداد استفاده شود
    for (int i = 0; i < 10; i++)
        for (int j = 0; j < 10; j++)
            v[i][j].push_back(twice( i*j )); این عبارت نمی‌تواند به عنوان یک پارامتر با ارجاع رد شود
    cout << v[0] << endl;
}
```

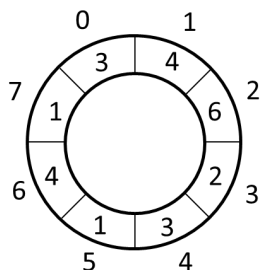
نوشتن بردارها به این صورت ممکن نیست

پرسش ۳ (۲ نمره) — فرض کنید یک تابع بازگشتی نوشته‌ایم که اجرای آن برای مقادیر بزرگ پارامترها بسیار کند است. به دو دلیل ممکن برای این کندی اشاره کنید. راهنمایی: یکی از این مشکلات در محاسبه جمله‌های دنباله فیبوناچی خود را نشان می‌دهد. (۱) زمان لازم برای کپی کردن پارامترها روی پشته (۲) وجود فراخوانی‌های تکراری

پرسش ۴ (۴ نمره) — تابعی بازگشتی بنویسید که دو رشته را دریافت کند و تعیین کند آیا رشته دوم زیردنباله‌ای از رشته اول است یا خیر. معنی زیردنباله این است که کاراکترهای رشته دوم به همان ترتیب در رشته اول ظاهر شوند، هر چند ممکن است بین آنها کاراکترهای دیگر بیایند. به عنوان مثال "bd" زیردنباله‌ای از "abcd" است. در نوشتن این تابع می‌توانید از s.length() برای یافتن طول رشته s و s.substr(1) برای به‌دست‌آوردن رشته‌ای استفاده کنید که از حذف کاراکتر اول s حاصل شده. در تابع شما استفاده از حلقه مجاز نیست.

```
bool subseq(string s1, string s2) {
    if (s2.length() == 0)
        return true;
    if (s1.length() == 0)
        return false;
    if (s1[0] == s2[0])
        return subseq(s1.substr(1), s2.substr(1));
    return subseq(s1.substr(1), s2);
}
```

پرسش ۵ (۴ نمره) — در یک بازی خیالی تعدادی عدد روی یک لیست حلقوی نوشته شده‌اند. در این بازی، یک خانه از لیست به عنوان خانه شروع و یکی به عنوان خانه هدف انتخاب می‌شوند. بازیکن در ابتدا در خانه شروع قرار دارد. در هر قدم، بازیکن به اندازه عددی که در خانه‌ای که قرار دارد به یکی از دو طرف جهت می‌کند. بازیکن باید بدون این که دو بار در یک خانه قرار بگیرد به خانه هدف برسد.



به عنوان مثال در شکل روبه‌رو (شماره خانه‌ها بیرون لیست نوشته شده‌اند)، اگر خانه شروع خانه شماره ۱ (حاوی عدد ۴) و خانه هدف خانه شماره ۷ باشد، بازیکن در ابتدا به سمت ساعت‌گرد ۴ خانه حرکت می‌کند و به خانه شماره ۵ می‌رود. بعد، یک خانه در جهت پادساعت‌گرد برمی‌گردد (خانه شماره ۴) و در نهایت سه خانه در جهت ساعت‌گرد حرکت می‌کند و به خانه مقصد می‌رسد. (البته راه‌های دیگری برای رسیدن به همین مقصد وجود دارد، که به ترتیب خانه‌های ۱، ۵، ۶، ۲، ۴ و ۷ را ملاقات می‌کند).

به روش پس‌گرد (backtracking)، تابعی بنویسید که یک بردار از اعداد که نماینده لیست حلقوی است را دریافت کند و در صورتی که بازی قابل حل باشد مقدار درست و در غیر این صورت مقدار نادرست را برگرداند. در نوشتن این تابع فرض کنید دو تابع کمکی `moveCW(pos, steps, size)` و `moveCCW(pos, steps, size)` داریم که اگر در لیستی حلقوی به اندازه `size` در خانه `pos` باشیم و به اندازه `steps` حرکت کنیم شماره خانه مقصد را در حرکت‌های ساعت‌گرد و پادساعت‌گرد برمی‌گردانند. به عنوان مثال `moveCCW(2, 6, 8)` مقدار ۴ را برمی‌گرداند. می‌توانید از این توابع استفاده کنید و نیاز نیست بدنه آنها را بنویسید. در تعریف فهرست پارامترها و استفاده از توابع کمکی آزاد هستید.

```
bool solve(vector<int> ring, int start, int dest) {
    if (start == dest)
        return true;
    int next_cw = move_cw(start, ring[start], ring.size());
    int next_ccw = move_ccw(start, ring[start], ring.size());
    ring[start] = 0;    // mark as visited
    if (ring[next_cw] != 0 && solve(ring, next_cw, dest))
        return true;
    if (ring[next_ccw] != 0 && solve(ring, next_ccw, dest))
        return true;
    return false;
}
```

پرسش ۶ (۴ نمره) — هدف این پرسش مدل‌سازی کارنامه تحصیلی دانشجویان است. کارنامه دانشجو شامل نام و شماره دانشجو به همراه سوابق درس‌های گرفته‌شده شامل نام درس، تعداد واحد و نمره درس می‌شود.

(الف) تایپ‌های لازم برای نگهداری کارنامه دانشجو را تعریف کنید. (بهتر است یک تایپ جداگانه برای نگهداری سوابق یک ترم تعریف کنید)

(ب) تابعی بنویسید که معدل کل دانشجو را حساب کند.

(ج) تابعی بنویسید که تعداد ترم‌های مشروط (معدل زیر ۱۲) دانشجو را حساب کند.

در تعریف تایپ‌ها و توابع به نکات زیر توجه نمایید:

- به هر تابع حداقل اطلاعات لازم پاس شود.
 - نام‌گذاری تایپ‌ها، توابع و متغیرها خوانا و معنی‌دار باشد. (اگر معنای بعضی واژه‌ها را نمی‌دانید به فینگلیش بنویسید.)
 - تا حد امکان از تکرار کدها پرهیز کنید. به این منظور می‌توانید توابعی اضافه بر آنچه خواسته شده تعریف کنید.
 - هیچ‌یک از توابع از ۵ جمله (statement) طولانی‌تر نباشد (شامل تعریف متغیرهای محلی).
- به عنوان نمونه، قطعه کد زیر ۴ جمله دارد:

```
int sum = 0;
for (int i = 0; i < v.size(); i++)
    if (v[i] > 10)
        sum += 10;

struct CourseRecord {
    string courseNo;
    double grade;
    int units;
};

struct TermRecord {
    string term;
    vector<CourseRecord> records;
};

struct Transcript {
    string name;
    string id;

    vector<TermRecord> terms;
};

double termPoints(TermRecord tr) {
    double totalPoints = 0;
    for (int i = 0; i < tr.records.size(); i++)
        totalPoints += tr.records[i].grade * tr.records[i].units;
    return totalPoints;
}

double termUnits(TermRecord tr) {
    int totalUnits = 0;
    for (int i = 0; i < tr.records.size(); i++)
        totalUnits += tr.records[i].units;
    return totalUnits;
}

int unitsPassed(Transcript t) {
```

```

    int totalUnits = 0;
    for (int i = 0; i < t.terms.size(); i++)
        totalUnits += termUnits(t.terms[i]);
    return totalUnits;
}

double calcGPA(Transcript t) {
    double totalPoints = 0;
    for (int i = 0; i < t.terms.size(); i++)
        totalPoints += termPoints(t.terms[i]);
    return totalPoints / unitsPassed(t);
}

double isMashroot(TermRecord tr) {
    return (termPoints(tr) / termUnits(tr)) < 12;
}

int mashrootCount(Transcript t) {
    int result = 0;
    for (int i = 0; i < t.terms.size(); i++)
        if (isMashroot(t.terms[i]))
            result++;
    return result;
}

```